

Database Management System Performance Comparisons

A Systematic Literature Review

Reading Notes & Industry Perspective

Based on: Taipalus (2024) - Journal of Systems and Software, Vol. 208

Document Summary / Reading Notes

Prepared by Abolfazl Rastgar Moghadam

LinkedIn: [linkedin.com/in/abolfazl-rastgar/](https://www.linkedin.com/in/abolfazl-rastgar/)

Relational	NoSQL	NewSQL	Benchmarking	Performance
------------	-------	--------	--------------	-------------

This document is a summary and reading note based on the Taipalus (2024) paper.

Prepared by Abolfazl Rastgar Moghadam

LinkedIn: [linkedin.com/in/abolfazl-rastgar/](https://www.linkedin.com/in/abolfazl-rastgar/)

Introduction

I recently came across a paper that I think is genuinely worth sharing with anyone who works with databases professionally - whether you are a DBA, a Data Engineer, a backend developer, or an architect responsible for technology decisions.

The paper is titled Database Management System Performance Comparisons: A Systematic Literature Review, authored by Toni Taipalus of the University of Jyväskylä, Finland, and published in the Journal of Systems and Software in February 2024 (Vol. 208, Article 111872).

What makes this paper different from the dozens of articles you find online comparing databases is that it does not compare databases directly. Instead, it systematically reviews and synthesizes the existing body of research that has attempted such comparisons - and the findings are quite sobering.

Performance is typically tested in very specific contexts which are not necessarily generalizable, and there are other aspects besides performance to consider.

- Taipalus (2024)

About the Paper

This is a Systematic Literature Review (SLR) - a rigorous academic methodology in which the author does not run experiments themselves, but instead collects, categorizes, and critically analyzes a large body of existing published research according to a defined protocol. The primary studies reviewed cover performance comparisons across Relational (RDBMS), NoSQL, and NewSQL database systems, published in peer-reviewed venues including ACM SIGMOD and VLDB.

The paper set out to answer five core questions:

- How has DBMS performance been tested in published research?
- How is performance recommended to be tested?
- How should performance comparison results be interpreted?
- What other aspects beyond performance should be considered?
- What research avenues would be most productive going forward?

Key Findings

After systematically reviewing the primary studies, the author identifies several critical patterns. These are not opinions - they are observations grounded in the aggregated evidence from published database performance research.

Finding 1 - Most Benchmarks Do Not Reflect Real-World Use Cases

This is the paper's most striking finding. The majority of performance comparison studies test databases in isolated, controlled environments with synthetic workloads - not in the messy, concurrent, multi-application environments that production systems actually operate in.

Benchmarks are typically run on a single machine, with the benchmark software and the DBMS on the same hardware, effectively eliminating network latency and real application behavior. This produces numbers that look clean and publishable, but tell you very little about what would happen if you deployed that same database in your environment.

Practical implication: When you read that 'MongoDB is X times faster than PostgreSQL' or 'Cassandra handles Y million writes per second,' ask yourself: under what conditions? With what data shape? With what consistency requirements? Against what workload profile? The numbers almost certainly do not translate directly to your situation.

Finding 2 - Results Are Reported with Insufficient Detail for Replication

A well-designed experiment must be reproducible. The paper finds that most DBMS performance comparison studies lack the level of methodological transparency needed to replicate their results or to draw strong conclusions from them. Key details - such as hardware specifications, exact configuration parameters, data distributions, and query optimization settings - are frequently omitted.

This matters significantly because DBMS performance is extremely sensitive to parameters such as available memory, buffer pool size, index configuration, and the ratio of read to write operations. Changing any of these can swing performance numbers by an order of magnitude.

Finding 3 - NoSQL Performance Gains Come with Real Trade-offs

The paper provides a clear-eyed technical explanation of why NoSQL systems often appear faster in benchmarks. It is not magic - it is a deliberate architectural trade-off.

Many NoSQL data models (particularly key-value, document, and wide-column stores) are designed around redundant data storage. Rather than normalizing data across tables and using joins, they duplicate data so that each query can be answered by reading from a single location. This makes individual queries fast and simple - but at the cost of storage space, increased write complexity, and the risk of data inconsistency when the same data exists in multiple places.

Dimension	RDBMS	NoSQL	NewSQL
Data Model	Relational tables	Key-value, document, graph, wide-column	Relational + distributed
Query Language	SQL standardized	DB-specific; no universal standard	SQL + extensions
Consistency	ACID strong consistency	BASE / eventual consistency	ACID at scale
Scalability	Primarily vertical	Horizontal via sharding / replication	Horizontal with ACID
Redundancy	Minimized through joins	High by design	Varies
Performance Strength	Complex queries, write integrity	High-throughput reads, simple queries	OLTP at scale
Typical Use	ERP, finance, core systems	Web apps, IoT, caching, graphs	Distributed OLTP, global apps

A NoSQL database that returns results faster in a benchmark may be doing so because it stored redundant copies of the data specifically shaped for that query. The performance gain is real in that narrow scenario - but the total cost of ownership includes larger storage, more complex write operations, and reduced data integrity guarantees.

Finding 4 - Specialized Databases Outperform General-Purpose Ones - In Their Domain

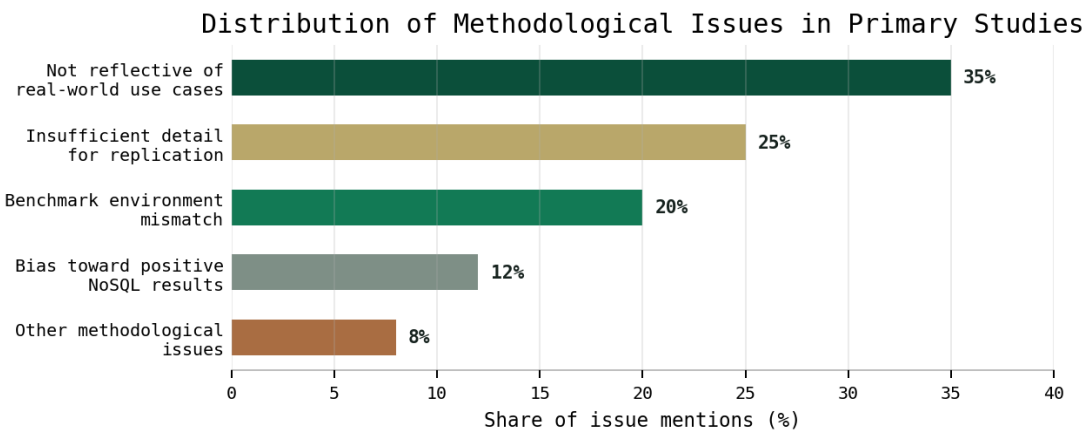
One of the most practically important observations in the paper is that a database system designed for a specific purpose will always outperform a general-purpose system - but only within that specific purpose.

A graph database like Neo4j will traverse complex relationship networks faster than PostgreSQL using recursive CTEs. A time-series database like TimescaleDB will compress and query sensor data more efficiently than a general-purpose document store. A columnar OLAP engine will scan billions of rows for aggregate queries faster than any row-oriented RDBMS.

But these comparisons are only meaningful within their intended domain. The mistake many organizations make is choosing a specialized system for its benchmark headline numbers, then discovering that their workload contains operations the specialized system handles poorly.

Finding 5 - Many Comparison Studies Show Bias Toward NoSQL

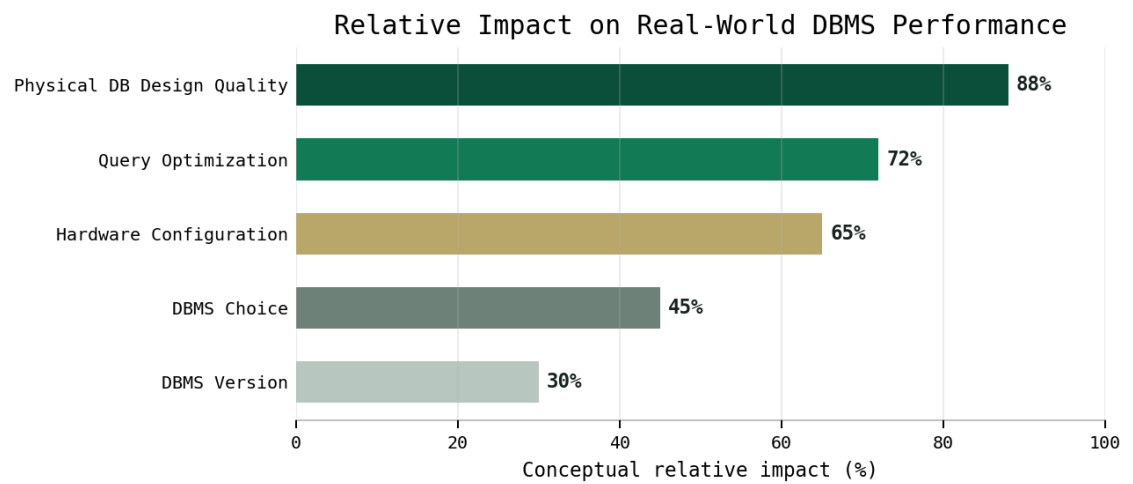
The paper makes a careful but direct observation: a notable portion of the primary studies reviewed contain praise for NoSQL systems with little or no critical analysis of their acknowledged weaknesses. This introduces a form of publication bias that distorts the collective picture that the literature paints.



Illustrative, based on the qualitative findings discussed in the paper.

Finding 6 - Physical Database Design Has More Impact Than DBMS Choice

Perhaps the most practically valuable finding in the entire paper is this: the physical design of your database - index strategy, schema structure, query patterns, partitioning approach - has a greater impact on real-world performance than which DBMS you choose.

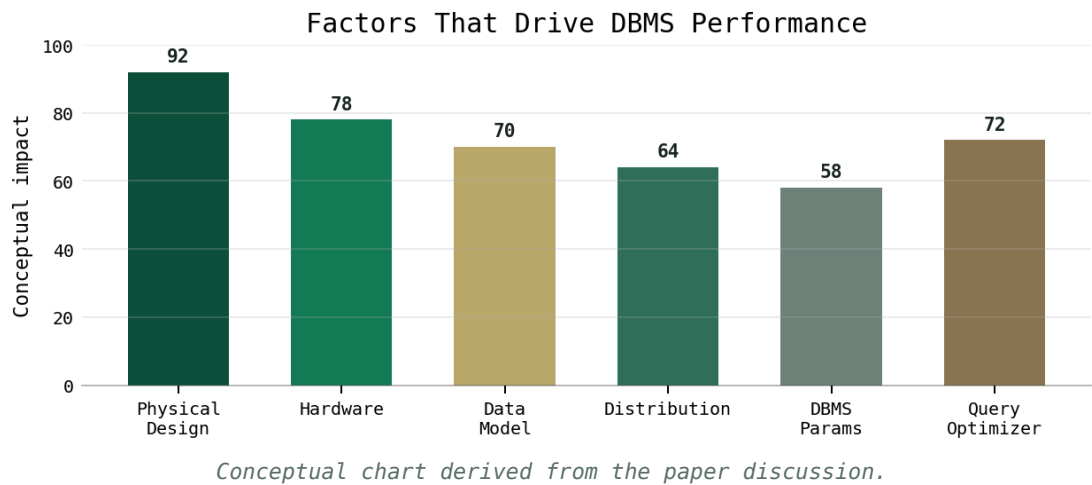


Conceptual visualization based on Taipalus (2024) discussion; it does not reproduce quantitative data from the original paper.

The main lesson is not that DBMS choice is irrelevant. The lesson is that DBMS choice rarely compensates for weak data modeling, poor indexing, unrealistic workloads, or missing operational discipline.

Factors That Drive DBMS Performance

The paper dedicates significant space to establishing a clear framework for understanding what actually drives database performance. These factors apply regardless of which type of DBMS you are working with.



Physical Database Design

The most impactful factor. In relational databases, this means index strategy, normalization decisions, partitioning, and ensuring the physical structures match the query patterns. In NoSQL systems, this means data model design - shaping data structures around query access patterns rather than normalized entity relationships. A poor physical design can make even the fastest DBMS perform terribly.

Hardware

Memory access is roughly four orders of magnitude faster than disk access. Every DBMS optimization strategy - buffer pools, caching, in-memory indexes - exists to minimize disk reads. The amount of RAM available to the DBMS fundamentally constrains its performance ceiling. Multi-core utilization also differs significantly between systems.

Data Model and Consistency Model

ACID transactions in relational systems introduce locking overhead that is absent in systems using eventual consistency. The choice between strong and eventual consistency is a direct performance trade-off - not a quality-of-implementation issue.

Distribution Architecture

Write operations in distributed systems face the CAP theorem directly. Every distributed DBMS must choose, on every write, between data consistency and data availability. This architectural constraint fundamentally shapes performance characteristics.

DBMS Parameters and Query Optimizer

Settings such as buffer pool size, work memory allocation, connection limits, and checkpoint frequency have direct performance implications. The quality of the query optimizer - how well it can select physical execution operators - varies significantly between systems and versions.

Conclusions from the Paper

The paper concludes with a set of recommendations for both industry practitioners and researchers. These conclusions follow directly from the findings and carry significant weight precisely because they are grounded in a systematic analysis of the literature rather than a single experiment.

Conclusion 1 - Benchmark Results Cannot Be Directly Applied to Your Environment

When a study states that one DBMS outperforms another, it has done so in a specific hardware, software, and workload configuration. That configuration is almost certainly different from yours. The paper is explicit: these results have limited relevance for deciding which DBMS is the best fit for your environment. Your environment is the only valid benchmark for your decision.

Conclusion 2 - Performance is Not the Only Criterion

The paper argues that focusing exclusively on performance when choosing a DBMS leads to poor decisions. Consistency guarantees, storage costs, operational complexity, query expressiveness, ecosystem maturity, and support availability all matter - and they cannot be measured by a throughput benchmark.

Conclusion 3 - Design Quality Matters More Than System Selection

The paper reinforces that performance bottlenecks are far more often a product of database design decisions than DBMS limitations. An optimally designed PostgreSQL schema will consistently outperform a poorly designed MongoDB deployment for the same workload.

Conclusion 4 - The Research Community Needs Better Benchmarking Standards

From an academic standpoint, the paper calls for more rigorous, reproducible, and realistic performance testing methodologies - with full disclosure of environment parameters, workload specifications, and optimization settings used.

From Academic Research to Industrial Practice

The author approaches these questions from a rigorous academic standpoint - systematic methodology, peer-reviewed publication, careful citation, and measured conclusions. But the findings translate with remarkable directness into practical guidance for anyone making real technology decisions.

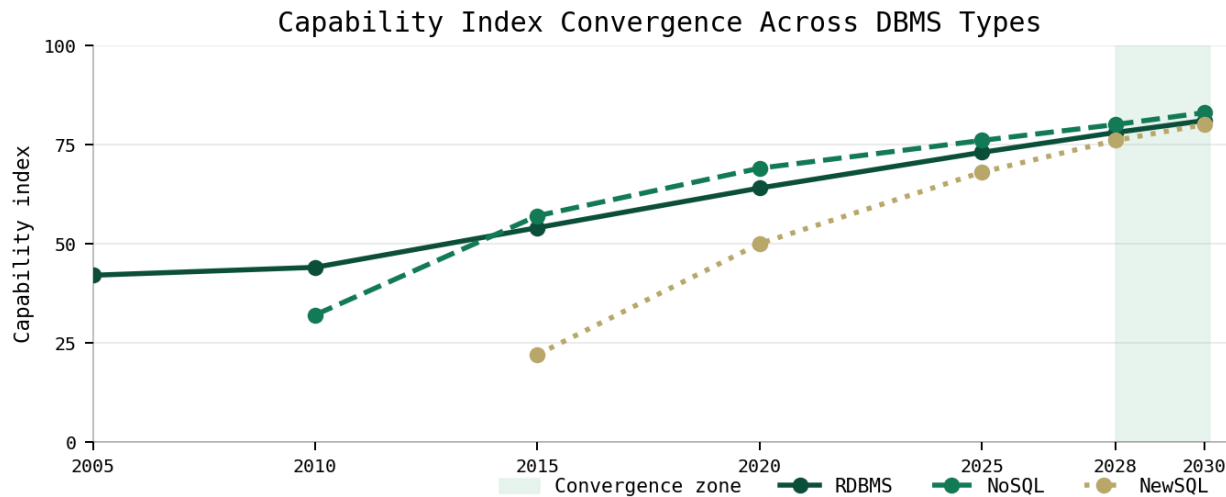
What the Academic Findings Mean in Practice

Academic Finding	Industrial Implication
Benchmarks do not reflect real-world conditions	Never make a database selection based primarily on published benchmark numbers.
NoSQL gains come from redundant data storage	Model your total cost: storage, write complexity, consistency management, and operational overhead.
Specialized DBMSs win in their specific domain	Define your workload precisely before evaluating any database system; the use case drives the choice.
Physical design dominates performance outcomes	Invest in schema design and index strategy first; do not migrate systems hoping for a performance fix.
Publication bias toward NoSQL results	Read comparison articles critically; seek vendor-neutral testing and real-world case studies.
Results lack replication detail	When evaluating a DBMS, run your own benchmark with your own data, queries, and hardware.

The paper does not tell you which database is best. It tells you that anyone who confidently tells you which database is best - based on generic benchmarks - is probably oversimplifying. The honest answer is always: it depends on your workload, your consistency requirements, your team, and your constraints.

The Convergence of Database Technologies

One of the most important trends in the database landscape - and one that the paper's findings point toward, even if not stated explicitly - is the accelerating convergence between database categories that were once considered fundamentally distinct.



Conceptual projection. The labels and convergence zone have been separated for readability.

Blurring Boundaries

PostgreSQL now supports native JSON and JSONB storage with indexing, making it a credible document store. It supports arrays, full-text search, graph-like queries with recursive CTEs, and time-series extensions. MongoDB has introduced multi-document ACID transactions. CockroachDB and YugabyteDB bring horizontal scalability to the relational model. Google Spanner proved that global distributed consistency at scale is achievable.

The hard boundaries that once made NoSQL clearly distinct from relational systems are eroding. Within the next few years, the performance and capability gaps that made database selection a clear either/or decision will continue to narrow significantly.

Convergence Themes

ACID Transactions in NoSQL MongoDB, DynamoDB, and Cassandra have all introduced varying forms of multi-document or multi-partition transaction support. The 'no ACID' argument for avoiding NoSQL in transactional contexts is becoming weaker.	Flexible Schema in RDBMS PostgreSQL's JSONB, SQL Server's JSON support, and MySQL's JSON column type all allow schema flexibility that was previously the exclusive territory of document databases.
Horizontal Scalability for SQL NewSQL systems and distributed RDBMS implementations such as CockroachDB, PlanetScale, and Neon are making SQL-compatible horizontal scaling increasingly accessible.	Unified Query Interfaces Multi-model databases that support document, graph, key-value, and relational access patterns within a single engine reduce the need to choose one paradigm upfront.

As capabilities converge, the performance arguments that once drove database selection will matter less. What will matter more - precisely as Taipalus (2024) argues - is design quality, workload understanding, and operational competence. The choice of DBMS will increasingly be a secondary decision.

The Most Important Factor: Database Design

In my view, the single most important takeaway from this paper - and from years of working with production database systems - is this:

A well-chosen database with poor design will destroy your system.

A good designer will lead you to the right database.

This is not an overstatement. The paper provides the academic evidence for what experienced practitioners have observed repeatedly in the field. Physical database design - the choices made about how data is structured, indexed, partitioned, and accessed - accounts for a far larger share of real-world performance outcomes than the database system itself.

A poorly indexed table in any database will cause full scans

Whether you are using PostgreSQL, MySQL, MongoDB, or Cassandra - if your queries cannot use an index to avoid scanning large volumes of data, performance will degrade. The DBMS cannot compensate for missing index coverage.

Schema design determines query complexity

A relational schema designed without considering the application's query patterns will require expensive multi-table joins. A NoSQL document model designed without understanding access patterns will force complex application-side data assembly. Good design starts with understanding how the data will be used, not just how it is structured.

Write patterns determine consistency overhead

The cost of maintaining ACID guarantees depends directly on how writes are structured. Short, targeted transactions are cheap. Long-running transactions with broad lock coverage are expensive. A good designer minimizes lock contention through careful transaction scoping.

A good designer chooses the right tool

Someone who understands workload characteristics, consistency requirements, access patterns, and scalability needs will naturally gravitate toward the appropriate database category. The selection is a design output, not a design input. Start with the requirements, not the technology.

Database Selection is Not a Purely Technical Decision

The paper focuses on the technical dimensions of database performance - benchmarks, data models, query execution, hardware, design. But it is important to acknowledge explicitly what the paper does not cover and what practitioners know from experience: the actual database selection decision in most organizations is as much organizational as it is technical.

Organizational Politics and Architecture Standards

Many organizations have enterprise architecture standards that dictate approved technology stacks. A team that wants to adopt a document database may face resistance from a central architecture board committed to relational systems - regardless of technical merit. Conversely, a hype-driven leadership team may push adoption of a trendy NoSQL system before the use case actually justifies it.

Developer Culture and Team Expertise

The most technically appropriate database is often not the optimal choice if your team has no experience with it. A team deeply skilled in PostgreSQL will outperform a team unfamiliar with Cassandra - even if Cassandra is theoretically better suited for the workload. Familiarity, tooling knowledge, operational muscle memory, and existing monitoring infrastructure all factor into the real-world effectiveness of any database choice.

Financial and Licensing Considerations

The cost dimension is frequently underweighted in technical evaluations. Oracle Database licensing costs can dwarf infrastructure costs. Cloud-managed database pricing models can produce unexpected bills at scale. Open-source systems have no licensing cost but carry operational overhead.

Vendor Support and Ecosystem Maturity

For production systems where data integrity and uptime are business-critical, the maturity of the support ecosystem - documentation, community size, availability of experienced contractors, managed cloud offerings, backup tooling, and monitoring integrations - is a real decision factor.

Regulatory and Compliance Requirements

Data residency laws, GDPR, HIPAA, SOC 2, and sector-specific regulations can eliminate certain database choices before any technical evaluation begins. A cloud-managed database that does not offer a region-specific deployment option in a required jurisdiction is simply not a viable option, regardless of performance characteristics.

A Practical Decision Framework

Drawing together the paper's findings with practical experience, here is a structured approach to database selection that reflects the lessons from Taipalus (2024):

1. Define Your Workload Before evaluating any database, characterize your workload precisely. What is the read/write ratio? Are writes transactional or bulk? What does a typical query look like? What are your latency requirements? What is your data volume today and in two years? What consistency guarantees does your business logic require?	2. Identify Non-Negotiable Constraints List the constraints that cannot be compromised: regulatory requirements, existing infrastructure, licensing budget, team skills, and deployment environment. These eliminate options before technical evaluation begins.
3. Design First, Select Second For your top candidate systems, design your data model and access patterns first. A good design often reveals which database category is most natural for your workload.	4. Benchmark Your Workload, Not Someone Else's If performance is a concern, build a representative benchmark using your actual data volume, queries, hardware, and consistency requirements. Do not rely on published benchmarks for this decision.
5. Account for Total Cost of Ownership Evaluate storage cost, operational complexity, monitoring tooling, backup strategy, team training requirements, and support availability - not just raw performance numbers.	6. Revisit the Decision as Technology Evolves Given the rapid convergence of database capabilities, a database selection made three years ago may be worth revisiting. The landscape is moving fast enough that previous assumptions may no longer hold.

Closing Thoughts

The Taipalus (2024) paper is the kind of research that should be required reading for anyone who participates in database technology decisions. Not because it provides definitive answers - it deliberately does not - but because it forces a more disciplined, evidence-based approach to a question that is too often answered with hype, tribal loyalty, or oversimplified benchmarks.

What I Take From This Paper

- The database debate - relational vs. NoSQL vs. NewSQL - is largely a false binary. Every category has genuine strengths and genuine weaknesses. The right answer depends on the question.
- Published performance benchmarks are evidence for the benchmark environment, not for your environment. Treat them as directional signals, not decision criteria.
- As database capabilities continue to converge, the quality of design and the depth of operational expertise will increasingly determine outcomes - not the system name.
- Database selection is a sociotechnical problem. Ignoring the organizational, financial, and cultural dimensions produces technically interesting decisions that fail in practice.
- The most valuable investment you can make in database performance is in the people and practices that produce good designs - not in migrating between systems.

Performance gains, such as those provided by many NoSQL systems, rely heavily on redundant data to minimize query complexity, providing faster response times. Naturally, storing redundant data increases storage cost and may lead to data inconsistencies.

- Taipalus (2024)

If this reading note has prompted a question about your own database design or selection process, the most useful next step is to read the original paper. The arXiv preprint is freely available at arxiv.org/abs/2301.01095. The peer-reviewed version is published in the Journal of Systems and Software, Vol. 208 (2024), Article 111872.

References

The following references include the primary paper reviewed in this document, along with key works cited within it that provide additional depth on the topics discussed.

Primary Reference

Taipalus, T. (2024). Database management system performance comparisons: A systematic literature review. *Journal of Systems and Software*, 208, Article 111872. <https://doi.org/10.1016/j.jss.2023.111872>

Taipalus, T. (2023). Database management system performance comparisons: A systematic literature review [Preprint]. arXiv. <https://arxiv.org/abs/2301.01095>

Key Works Cited in the Paper

Davoudian, A., Chen, L., & Liu, M. (2018). A survey on NoSQL stores. *ACM Computing Surveys*, 51(2), 1-43. <https://doi.org/10.1145/3158661>

Pavlo, A., & Aslett, M. (2016). What's really new with NewSQL? *ACM SIGMOD Record*, 45(2), 45-55.

Corbett, J. C., et al. (2013). Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems*, 31(3), 1-22.

Hellerstein, J. M., Stonebraker, M., & Hamilton, J. (2007). Architecture of a database system. *Foundations and Trends in Databases*, 1(2), 141-259.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387.

Haerder, T., & Reuter, A. (1983). Principles of transaction-oriented database recovery. *ACM Computing Surveys*, 15(4), 287-317. [Original ACID paper]

Grolinger, K., et al. (2013). Data management in cloud environments: NoSQL and NewSQL data stores. *Journal of Cloud Computing*, 2(1), 1-24.

Raasveldt, M., et al. (2018). Fair benchmarking considered difficult: Common pitfalls in database performance testing. *DBTest*, ACM SIGMOD.

Difallah, D. E., Pavlo, A., Curino, C., & Cudre-Mauroux, P. (2013). OLTP-Bench: An extensible testbed for benchmarking relational databases. *PVLDB*, 7(4), 277-288.

Further Reading

Gessert, F., Wingerath, W., Friedrich, S., & Ritter, N. (2017). NoSQL database systems: A survey and decision guidance. *Computer Science - Research and Development*, 32(3-4), 353-365.

Khasawneh, T. N., AL-Sahlee, M. H., & Safia, A. A. (2020). SQL, NewSQL, and NOSQL databases: A comparative survey. *ICICS*, IEEE, pp. 013-021.

Khan, W., et al. (2023). SQL and NoSQL database software architecture performance analysis and assessments - A systematic literature review. *Big Data and Cognitive Computing*, 7(2), 97.

About This Document

This document is a concise summary and set of personal reading notes prepared by Abolfazl Rastgar Moghadam, based on the systematic literature review published by Toni Taipalus in the Journal of Systems and Software (2024). LinkedIn: [linkedin.com/in/abolfazl-rastgar/](https://www.linkedin.com/in/abolfazl-rastgar/). It is intended for educational and professional sharing purposes.

Document Structure Summary

Section	Pages	Content
Introduction	2	Context and motivation for reading the paper
Key Findings	3-6	Six major findings from the systematic review
Performance Factors	7	Technical framework: what drives DBMS performance
Conclusions	8	Paper's recommendations for industry and research
Academic vs. Industry	9	Translation of findings into practical guidance
Technology Convergence	10-11	Forward-looking analysis of database evolution
Design is Everything	12	The primacy of physical database design
Non-Technical Factors	13	Organizational, financial, and cultural dimensions
Decision Framework	14	Six-step practical approach to database selection
Closing Thoughts	15	Summary takeaways and key quote
References	16	Full citation list including paper and cited works

Disclaimer: The analysis, interpretations, and opinions expressed in this document - beyond direct summaries of the paper's stated findings - represent the personal professional perspective of the author of these reading notes. Diagrams and charts are conceptual illustrations based on the paper's qualitative discussion and do not reproduce quantitative data from the original publication. All intellectual property of the original paper belongs to its author and publisher.

Database design is not a technical detail.
It is a strategic capability.